DEEP LEARNING-BASED MALWARE CLASSIFICATION

COURSE:

CIS*6530 CYBER THREAT INTELLIGENCE AND ADVERSARIAL RISK ANALYSIS

SUBMITTED BY:

SUPRIM DEVKOTA

KHUSHBUKUMARI HEMALKUMAR PATEL

APRIL, 2026

Contents

1. Abstract

Malware attribution is a critical challenge in cyber threat intelligence, where machine learning techniques can be applied to identify malicious software and associate it with Advanced Persistent Threat (APT) groups based solely on static program features. This study adapts the deep Convolutional Neural Network (CNN) architecture proposed by McLaughlin et al. [1], originally designed for binary Android malware detection, and extends it to multi-class APT group attribution using opcode sequences extracted from statically disassembled executables. To prevent data leakage and produce unbiased performance estimates, a nested cross-validation framework is adopted, with vocabulary construction and hyperparameter selection confined strictly to each outer training partition. The performance of the CNN model is compared against traditional machine learning classifiers established in the prior phase of this study, evaluating whether end-to-end deep learning offers a representational advantage over n-gram features for opcode-based APT group attribution.

2. Introduction

In the previous phase of this study, opcode-based malware classification was performed using traditional machine learning models such as Support Vector Machines (SVM), K-Nearest Neighbours (KNN), and Decision Trees. These models relied on n-gram feature representations, where opcode sequences were transformed into statistical patterns based on individual instructions and pairs of consecutive instructions. While this approach enabled effective baseline classification, it introduced several limitations.

Despite achieving meaningful discrimination, traditional n-gram-based approaches struggle to capture the full sequential and hierarchical structure of opcode streams. Unigrams treat each opcode independently, ignoring inter-instruction relationships, while bigrams capture only limited local dependencies. Furthermore, aggregated statistical representations inherently discard positional and contextual information critical for identifying complex malware behaviour, and their reliance on manual feature engineering limits adaptability to new or evolving malware variants.

To overcome these limitations, this study explores the use of deep learning, specifically one-dimensional Convolutional Neural Networks (1D-CNNs), for opcode-based malware classification. CNNs operate directly on sequential data and automatically learn hierarchical feature representations from raw opcode sequences, eliminating the need for manual feature engineering and offering greater scalability and adaptability compared to statistical n-gram methods.

The primary objective of this study is to evaluate whether a CNN-based architecture can outperform traditional machine learning approaches for malware classification using only opcode sequences. By maintaining consistency with the dataset and evaluation methodology from the previous phase, this study enables a direct and fair comparison between classical and deep learning approaches, providing insights into the effectiveness of learned representations for opcode-based APT group attribution.

3. Related Theory

This section outlines the core theories and methods behind the malware classification system, covering opcode representation, embedding, CNN-based feature extraction, and evaluation metrics.

3.1 Malware Classification

Malware classification is the automated process of categorizing malicious software specimens into predefined family or type labels based on their static or dynamic properties. Static analysis methods operate directly on the binary or disassembly representation of a program without execution, while dynamic approaches observe runtime behaviour in a controlled environment [2]. Supervised machine learning formulations treat malware classification as a multi-class problem, in which a model is trained to assign each sample to exactly one of K mutually exclusive malware families [3].

3.2 Opcode Sequence Representation

An opcode is the mnemonic component of a machine instruction specifying the fundamental operation to be performed by the processor. When a binary executable is disassembled, its instruction stream is represented as an ordered sequence of opcode tokens, constituting a structural fingerprint of the program's logic [4]. This representation captures the syntactic structure of execution flow and has been established as a discriminative feature space for malware family identification [5].

3.3 Token Embedding

A token embedding is a dense, low-dimensional real-valued vector representation of a discrete symbol, learned through a parameterised embedding matrix. Each token is assigned a unique trainable vector such that functionally related tokens tend to occupy proximate regions of the embedding space [6]. Unlike sparse one-hot encodings, embeddings produce compact continuous representations that serve as informative inputs to downstream neural network layers [7].

3.4 One-Dimensional Convolutional Neural Network

A Convolutional Neural Network (CNN) is a deep neural network architecture that applies learnable filter kernels over structured input to extract local, spatially invariant features

through shared-weight convolution [8]. The one-dimensional variant operates along a single sequential axis, making it well-suited to sequence-structured inputs such as text and op-code streams [9]. Stacking multiple convolutional layers enables hierarchical composition of progressively abstract features.

3.5 Rectified Linear Unit (ReLU)

The Rectified Linear Unit (ReLU) is a piecewise-linear activation function that passes positive input values unchanged while mapping all negative values to zero [10]. It is the standard nonlinear activation in contemporary deep neural networks, valued for its computational simplicity and its role in mitigating the vanishing gradient problem that affects saturating activations such as sigmoid and hyperbolic tangent functions [10].

3.6 Global Max Pooling

Global max pooling is a dimensionality reduction operation that collapses a variable-length feature map into a single scalar per filter channel by retaining only the maximum activation across all temporal positions [11]. Applied after convolutional layers, it produces a fixed-length representation invariant to input sequence length, functioning as a detector of whether a particular learned pattern was present anywhere in the input [9].

3.7 Cross-Entropy Loss with Class Weighting

Cross-entropy loss is the standard objective function for probabilistic multi-class classification, measuring the divergence between the predicted class probability distribution and the true label distribution [12]. When training data is class-imbalanced, inverse-frequency class weighting rescales each class's contribution to the loss proportionally to the inverse of its frequency, counteracting the tendency of unweighted loss to bias learning toward majority classes [13].

3.8 Adam Optimiser

Adam (Adaptive Moment Estimation) is a stochastic gradient-based optimisation algorithm that maintains per-parameter adaptive learning rates by computing exponentially decaying averages of both the gradient and the squared gradient [14]. It combines the adaptive properties of RMSProp with momentum-based updates and has become a widely adopted default optimiser for training deep neural networks [14].

3.9 Nested Cross-Validation

Nested cross-validation employs two concentric loops of k-fold cross-validation to produce an unbiased estimate of generalisation performance while simultaneously performing hyper-

parameter selection [15]. The outer loop provides held-out test folds never exposed during model selection; the inner loop operates exclusively on the outer training partition for hyperparameter optimisation. This strict separation eliminates the optimistic bias that arises when the same data is used for both selection and evaluation [16].

3.10 Stratified k-Fold Cross-Validation

Stratified k-fold cross-validation partitions the dataset into k folds such that the class label distribution within each fold mirrors that of the full dataset [15]. This is essential in multi-class settings with imbalanced class frequencies, where random partitioning may produce folds with absent or severely underrepresented classes, introducing bias into both training and evaluation [15].

3.11 Random Hyperparameter Search

Random hyperparameter search samples candidate configurations uniformly at random from a predefined search space and evaluates each against a validation criterion [17]. Bergstra and Bengio [17] demonstrated that random search outperforms grid search when model performance is sensitive to only a subset of hyperparameters, as random sampling achieves denser coverage of influential dimensions per unit of computational budget.

3.12 Classification Evaluation Metrics

Standard classification performance is characterised by four complementary metrics. Accuracy is the proportion of all samples correctly classified. Precision is the fraction of positive predictions for a class that are correct. Recall is the fraction of true instances of a class that are successfully retrieved. The F1-score is the harmonic mean of precision and recall, providing a balanced summary robust to class imbalance. In multi-class settings, weighted averaging aggregates each metric by class support to accommodate unequal class frequencies [18].

3.13 Confusion Matrix

A confusion matrix is a square matrix in which each entry records the number of samples of a given true class assigned to each predicted class [18]. Diagonal entries represent correct classifications; off-diagonal entries expose specific misclassification patterns. Row-normalisation yields per-class recall on the diagonal and reveals the directional structure of inter-class confusion, providing a diagnostically richer summary than aggregate scalar metrics alone [18].

4. Experimental Setup

The experimental workflow was conducted within the Kaggle computational environment, which provides a standardized and isolated runtime for machine learning experimentation. This environment includes preconfigured system libraries, Python dependencies, and deep learning frameworks, thereby eliminating inconsistencies associated with local system configurations. The use of a managed execution platform ensures uniformity across experimental runs and simplifies dependency management.

Model training and evaluation leveraged 2 NVIDIA Tesla T4 GPUs available within Kaggle to accelerate computation. GPU acceleration was essential for efficiently training the Convolutional neural network (CNN) architecture on high-dimensional opcode sequence representations, enabling faster convergence and supporting iterative experimentation during hyperparameter tuning.

The dataset used in this study corresponds to a preprocessed dataset generated in a prior phase (Submission 4) using the `preprocess.py` script. To ensure consistency across experimental stages, the complete preprocessed dataset directory was uploaded to the Kaggle environment and reused without modification. This approach guarantees that all experiments operate on an identical feature space and data distribution, thereby eliminating variability introduced by repeated preprocessing.

All output artifacts, including trained model parameters, evaluation logs, and intermediate results, were stored within the kaggle working directory. This directory serves as the default writable workspace in Kaggle and enables systematic organization and retrieval of experimental outputs for subsequent analysis.

Model execution was performed within the Kaggle notebook interface, where the—`model_neural.py` script was run as part of the notebook workflow. For completeness and portability, the equivalent standalone execution of the training pipeline can be invoked using:

Execution Command

```
python model_neural.py
```

Although the experiments were conducted in a cloud-based environment, the training script remains compatible with local deployment. Reproduction in a local setup requires access to a CUDA-enabled GPU, sufficient memory resources to support model training,

and a properly configured software stack with dependencies aligned to the Kaggle runtime.

To ensure reproducibility, all experiments were performed using a fixed dataset snapshot, a consistent runtime configuration, and an identical execution pipeline across runs. The use of a preprocessed dataset, combined with the controlled Kaggle environment, minimizes sources of variability and supports reliable replication of results.

5. Methodology

The malware classification process begins by importing the preprocessed opcode dataset obtained in the previous stage of the project (Submission 4). This dataset is first used to construct a vocabulary representing the unique opcodes present in the samples. The opcode sequences are then provided as input to a one-dimensional Convolutional Neural Network (1D CNN), which is trained to classify malware samples according to their associated Advanced Persistent Threat (APT) groups.

Measures are taken to prevent data leakage through the use of nested cross-validation. Additionally, strategies are implemented to address class imbalance within the dataset. A systematic hyperparameter search and tuning process is also conducted to determine the optimal configuration of the model's hyperparameters, thereby improving overall classification performance.

5.1 Preprocessed Dataset

The imported dataset consists of preprocessed opcode sequences, where addresses and operands have been removed, along with labels indicating the associated APT group for each malware sample. Specifically, the following files are imported and subsequently utilized throughout the modeling pipeline:

- `samples.json` – contains the preprocessed opcodes for all samples in the dataset.
- `y.npy` – stores the APT group label for each sample.
- `class_names.npy` – maps the numeric labels to actual APT group names.

These files collectively represent a total of 261 samples, which are categorized according to the following distribution.

Table 5.1: Imported APT Group Sample Counts

Name	Samples	Name	Samples	Name	Samples	Name	Samples
G0008_Carbanak	7	G0033_PoseidonGroup	5	G0068_PLATINUM	7	G0098_BlackTech	9
G0012_DarkHotel	7	G0038_Stealth Falcon	17	G0076_Thrip	6	G0107_Whitefly	6
G0020_Equation	12	G0041_Strider	11	G0081_Tropic Trooper	6	G0126_Higaisa	5
G0026_APT18	5	G0044_Winnti Group	6	G0082_APT38	8	G0128_ZIRCONIUM	5
G0027_Threat Group-3390	5	G0056_PROMETHIUM	17	G0094_Kimsuky	19	**Other**	34
G0030_Lotus Blossom	9	G0067_APT37	18	G0095_Machete	6		
G0031_Dust Storm	8						
G0032_Lazarus Group	23						

5.2 Vocabulary Building

Before any numeric processing can occur, these symbolic tokens must be mapped to integer indices through a vocabulary, which is a finite dictionary that enumerates every unique opcode observed in the training corpus.

The vocabulary is built by iterating over every token in every training sequence and collecting the complete set of unique opcodes via a frequency counter. Two special tokens are prepended at fixed positions before the opcode list is added:

- $\langle$ PAD $\rangle$ at index 0 — a dummy token used to fill sequences that are shorter than the fixed-length input the model requires.
- $\langle$ UNK $\rangle$ at index 1 — a catch-all token for any opcode encountered at inference time that was not present in the training corpus.

The final vocabulary is thus:

$$[\langle \text{PAD} \rangle, \langle \text{UNK} \rangle, \text{opcode}_1, \text{opcode}_2, \ldots, \text{opcode}_V]$$

where V is the number of unique opcodes seen across all training sequences.

Critically, the vocabulary is rebuilt independently for each outer fold using only that fold's training sequences. This is a deliberate design decision to prevent data leakage — if the vocabulary were built from the full dataset including test sequences, the model would implicitly know which opcodes appear in the test files before it ever sees them. By restricting vocabulary construction to training data, test-fold opcodes that happen not to appear in training are degraded to $\langle$ UNK $\rangle$ at encoding time, which accurately reflects the uncertainty a deployed model would face when encountering a previously unseen instruction.

Once the vocabulary is established, each sequence is encoded by replacing every token with its corresponding integer index. Since neural networks require fixed-size inputs, all sequences are then either truncated or zero-padded to a uniform length of `MAX_SEQ_LEN` = 50,000 tokens. Truncation removes tokens beyond position 50,000 and padding fills any remaining positions with `PAD_IDX` = 0. The result is a two-dimensional integer matrix of shape (N, 50000) where N is the number of samples, which is then converted to a PyTorch tensor for batched training.

5.3 Model Architecture

The classifier is a one-dimensional Convolutional Neural Network (CNN) that operates directly on the integer-encoded opcode sequences. The architecture consists of four sequential

stages: an embedding layer, a convolutional block, global max pooling, and a two-layer classification head.

5.3.1 Embedding Layer

The integer-encoded sequence is first passed through a trainable embedding layer which maintains a learnable matrix of shape `(vocab_size, embed_dim)` where each row is a dense vector representation of one vocabulary token. At forward pass time, each integer index in the input sequence is replaced by its corresponding row vector, transforming the input from shape `(B, L)` — where B is batch size and L is sequence length — to shape `(B, L, embed_dim)`. The `padding_idx`=0 argument ensures that ⟨ PAD ⟩ tokens always produce a zero vector and contribute no gradient during backpropagation. The tensor is then transposed to shape `(B, embed_dim, L)` to conform to PyTorch's convention for 1D convolution, which expects channels in the second dimension.

5.3.2 1D Convolutional Block

The embedded sequence passes through a convolutional layer followed by a ReLU activation. A 1D convolution slides a bank of `n_filters` learnable filters of width `filter_size` along the sequence dimension. Each filter computes a dot product between its weights and a local window of consecutive opcode embeddings, producing a scalar activation that represents the presence of a particular local pattern at that position. The output is therefore a feature map of shape `(B, n_filters, L)`, where each of the `n_filters` channels encodes the response of one learned pattern detector across every position in the sequence. The `padding=filter_size // 2` argument applies zero-padding to the sequence boundaries so that the output length remains equal to the input length regardless of filter size — known as same padding. A larger filter size captures longer-range opcode patterns while a smaller size is more sensitive to short local motifs.

5.3.3 Global Max Pooling

After the convolutional block, global max pooling is applied across the entire sequence dimension.For each of the `n_filters` feature maps, the single largest activation value across all sequence positions is retained. This reduces the variable-length feature map tensor from shape `(B, n_filters, L)` to a fixed-size vector of shape `(B, n_filters)`, regardless of the original sequence length. Global max pooling is particularly well-suited to malware classification because it answers the question "does this pattern appear anywhere in the executable?" rather than requiring the pattern to appear at a specific position — reflecting the reality that a suspicious instruction sequence is significant wherever it occurs in the binary.

5.3.4 Classification Head

The pooled feature vector passes through a two-layer fully connected classification head:

$$\text{nn.Linear}(n\_filters,\ hidden\_dim) \ \rightarrow\ \text{ReLU} \ \rightarrow\ \text{nn.Linear}(hidden\_dim,\ n\_classes)$$

The first layer projects the `n_filters`-dimensional pooled representation into a `hidden_dim`-dimensional space, followed by a `ReLU` non-linearity to introduce additional representational capacity.

The second linear layer projects this to `n_classes` output logits — one per APT Group and log-softmax is applied internally for numerical stability.

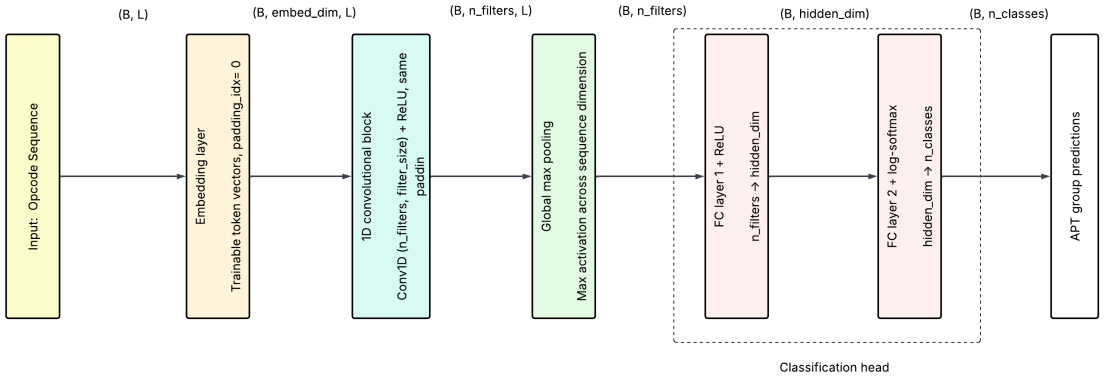

Figure 5.1: Model architecture diagram

5.4 Loss Function and Class Balancing

The model is trained by minimising a class-weighted cross-entropy loss. Because the dataset is class-imbalanced, each class is assigned a weight inversely proportional to its frequency in the training fold. These weights are passed to `nn.CrossEntropyLoss`, causing the gradient contributions of minority classes to be upscaled relative to majority classes, preventing the model from converging to a solution that simply predicts dominant families. The model is optimised using Adam.

5.5 Data Leakage Considerations

Data leakage occurs when information from outside the legitimate training set influences the model during learning or evaluation, producing performance estimates that are optimistically biased and fail to generalise to truly unseen data. In sequence classification pipelines like this one, leakage can occur at multiple stages and is not always obvious. Two specific leakage risks were identified and addressed in this implementation.

5.5.1 Vocabulary Leakage

A subtle leakage risk in opcode-based classification concerns vocabulary construction. If the token-to-index mapping were derived from the full dataset before any splitting occurred, the model would implicitly benefit from knowing which opcodes appear in test executables — for instance, a rare opcode appearing exclusively in test samples would receive a dedicated vocabulary entry rather than being degraded to ⟨ UNK ⟩. While this does not directly expose test labels, it constitutes a form of distributional leakage: the representation of the training data is shaped by test data, which would not be possible in a real deployment scenario where test samples are by definition unseen at training time.

To eliminate this, the vocabulary is constructed strictly within each outer training fold and applied to the corresponding test fold at encoding time. Test-fold opcodes absent from the training vocabulary are mapped to⟨ UNK ⟩, faithfully simulating the conditions of deployment on novel executables.

5.5.2 Hyperparameter Leakage

A well-known form of leakage in model development is hyperparameter leakage — the inadvertent use of test data to guide model selection. If hyperparameters are tuned by evaluating candidate configurations directly on the test fold, the test fold effectively becomes part of the training process, and the resulting performance estimate reflects the best achievable score on that particular split rather than expected performance on unseen data.

This is addressed through the nested cross-validation design described in the section below.

5.6 Nested Cross Validation

Nested cross-validation introduces two separate loops with distinct responsibilities, ensuring that no single data partition is ever used for both model selection and model evaluation.

5.6.1 Outer Loop – Unbiased Performance Estimation

The outer loop partitions the full dataset into `FINAL_FOLDS = 5` stratified folds, where stratification ensures that the class distribution is preserved proportionally in each fold. In each iteration, one fold is designated as the outer test set and the remaining four folds form the outer training set. The outer test set is entirely withheld from all downstream processing, including vocabulary construction, hyperparameter search, and final model training, until the moment of evaluation. This makes the outer test fold a genuine proxy for unseen deployment data, and aggregating predictions across all five outer test folds produces an unbiased estimate of generalisation performance.

5.6.2 Inner Loop – Hyperparameter Selection

Within each outer fold, hyperparameter selection is performed exclusively on the outer training data through a second, independent cross-validation procedure. A random search evaluates all `N_TRIALS = 64` candidate configurations from the discrete search space. Each configuration is assessed via `TUNE_FOLDS = 3`-fold stratified cross-validation run for `TUNE_EPOCHS = 20` epochs, producing a mean weighted F1 score across the three inner folds. The configuration achieving the highest inner F1 is selected as the best configuration for that outer fold.

Because the inner loop never accesses the outer test fold, the hyperparameter selection process is fully contained within the outer training partition. The vocabulary used during inner CV is also derived solely from the outer training data, maintaining the leakage controls described above even within the tuning stage.

5.6.3 Final Training and Evaluation

Once the best configuration is identified through inner CV, it is used to train a fresh model on the complete outer training set for `FINAL_EPOCHS = 50` epochs. This trained model is then evaluated on the outer test fold, producing predictions that contribute to the aggregated final metrics. This process is summarised as follows:

Algorithm 1 Nested Cross-Validation Training Procedure

for $k = 1$ to 5 **do**
  Hold out outer test set
  Use remaining data as outer train set
  Build vocabulary from outer train set
  for each of the 64 hyperparameter configurations **do**
    Perform 3-fold inner cross-validation (20 epochs)
    Record mean weighted F1 across inner folds
  end for
  Select configuration with highest mean inner weighted F1
  Train final model on full outer training set (50 epochs)
  Evaluate model on outer test set
  Store predictions
end for

5.7 Hyperparameter tuning configurations

For each outer fold, 64 random hyperparameter configurations are sampled from the defined search space and evaluated using inner cross-validation. In total, six hyperparameters are examined during this process.

Hyperparameter	**Options**
Embedding dimension	16, 32
Number of filters	64, 128
Filter (kernel) size	16, 32
Hidden layer size	64, 128
Learning rate	0.01, 0.001
Batch size	16, 32

Table 5.2: Hyperparameter options for the model.

5.8 Key Design Choices / Deviations

The proposed architecture is a faithful single-layer implementation of the design described in the reference [1]. Certain adaptations were required to align the model with the specific requirements of this work, and these design choices are detailed below:

- **Single vs. Multiple Convolutional Layers:** The original reference proposes a design with L stacked convolutional layers[1]. However, the model implemented in this study employs a single convolutional block. This simplification is justified by the observation in the reference that a single layer is ultimately used to mitigate overfitting[1].
- **Output Logits vs. Softmax Probabilities:** The original reference applies a softmax at the output to produce class probabilities[1]. In this model, raw logits are used instead, as PyTorch's `nn.CrossEntropyLoss` combines softmax with cross-entropy internally, yielding the same training behavior more efficiently.
- **Classification Target:** The original reference is framed as binary malware/benign classification[1]. However, this model targets multi-class APT Group attribution.
- **Input Representation:** The original reference encodes opcodes as one-hot vectors before the embedding lookup [1]. This model uses integer indices directly with `nn.Embedding`, which is algebraically equivalent but computationally more efficient.

6. Results & Analysis

The hyperparameters reported in [1] were initially applied, but the resulting performance was extremely low, indicating the need for tuning. Six hyperparameters were therefore optimized to identify the best configuration. The final model was then trained and evaluated, and its performance compared with the SVM, KNN, and Decision Tree results from the previous submission.

6.1 Initial results

A single end-to-end MalwareCNN model was trained via backpropagation using the hyperparameters specified in [1]. These include an 8-dimensional embedding space, 64 convolutional filters with a kernel size of 8, a fully connected layer with 16 neurons, a learning rate of 0.01, a batch size of 16, and 10 training epochs. However, the resulting performance metrics were extremely low, indicating the need to tune the hyperparameters.

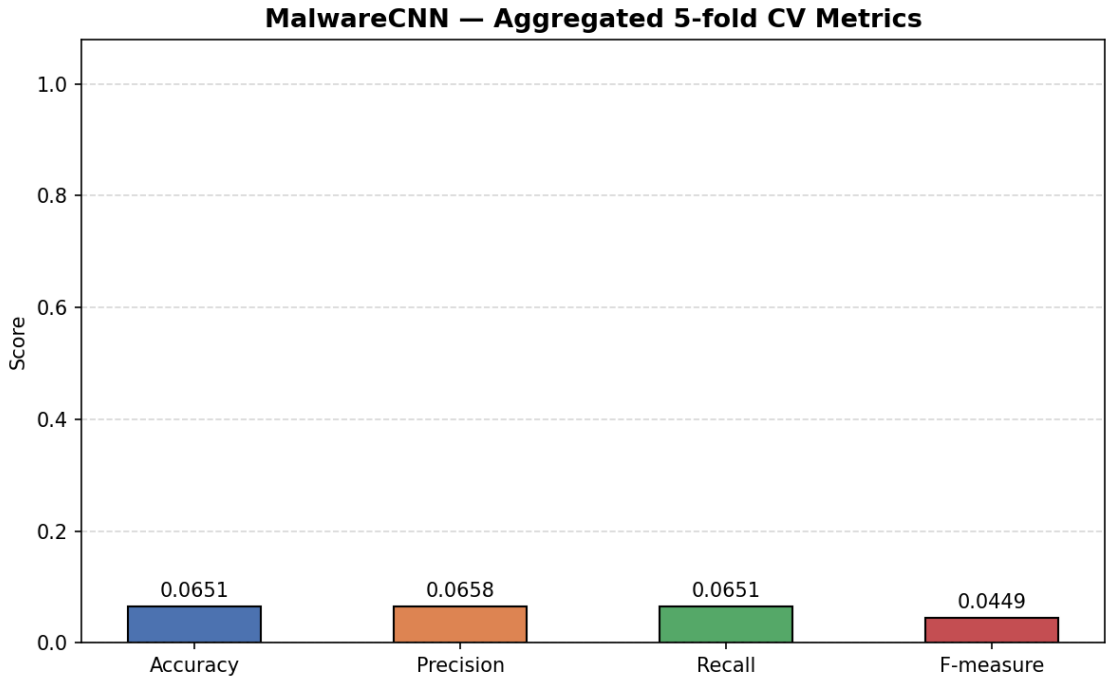

Figure 6.1: Performance metrics obtained using reference's hyperparameters

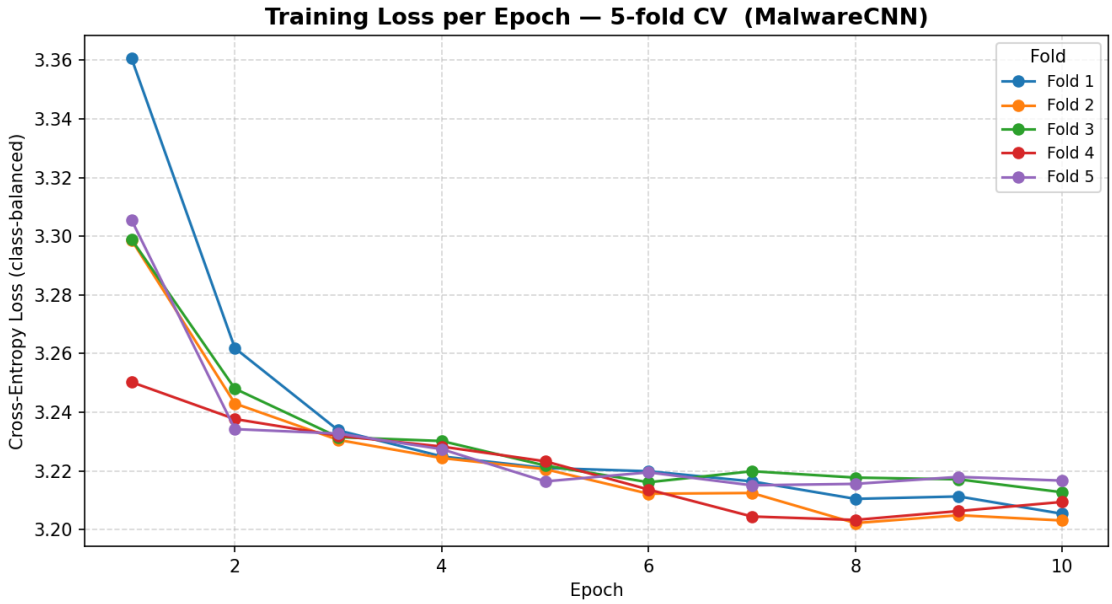

Figure 6.2: Training loss curve per epoch

The poor performance may be attributed to several factors. The use of vanilla SGD for the learning process without momentum can slow convergence, particularly in sparse embedding spaces. Additionally, training for only 10 epochs is likely insufficient when the embedding layer is learned from scratch. The small hidden dimension (16 neurons) also creates a representational bottleneck between the convolutional filters and the output classes. Finally, the larger opcode vocabulary in this dataset, compared to the 218 Dalvik opcodes used in [1], increases model complexity and learning difficulty.

6.2 Hyperparameter Tuning

Due to the poor performance observed with the original hyperparameters, tuning was performed on six hyperparameters (Section 5.7) to identify an improved configuration and analyze their effect on the inner F-measure.The key observations are as follows:

- **Embedding Dimension (16 vs. 32)**: Increasing the embedding dimension from 16 to 32 yields a moderate improvement in median F-measure but a noticeably wider interquartile range (IQR), suggesting that larger embeddings improve performance but destabilize it. This makes embedding dimension a moderately influential hyperparameter.

- **Number of Filters (64 vs. 128)**: The two values produce nearly identical median F-measures but IQR is noticeably wider for 128 filters, indicating that this hyperparameter has minimal marginal impact within the tested range. The model does not appear to benefit from doubling the number of convolutional filters.

- **Filter Size (16 vs. 32)**: A larger filter size shifts the median upward and widens the IQR slightly, mirroring the pattern observed for embedding dimension. This suggests that larger receptive fields capture more useful features for this task, though the effect size is modest.

- **Hidden Dimension (64 vs. 128)**: The transition from 64 to 128 hidden units produces a small upward shift in the median with a reduced IQR. The effect is real but marginal, implying diminishing returns from simply widening the hidden layer within this range.

- **Learning Rate (0.001 vs. 0.01)**: This is the most influential hyperparameter observed. A learning rate of 0.001 yields a substantially higher median F-measure and a tighter IQR compared to 0.01, where the distribution is more dispersed and the median drops noticeably. This strongly suggests that a learning rate of 0.01 is too aggressive for this architecture, leading to inconsistent convergence. The learning rate should be prioritized in any further tuning.

- **Batch Size (16 vs. 32)**: The two batch sizes produce nearly identical distributions in median but a noticeably tighter IQR for a batch size of 32, indicating that the model is largely insensitive to this parameter within the tested range but increasing batch size increases stability.

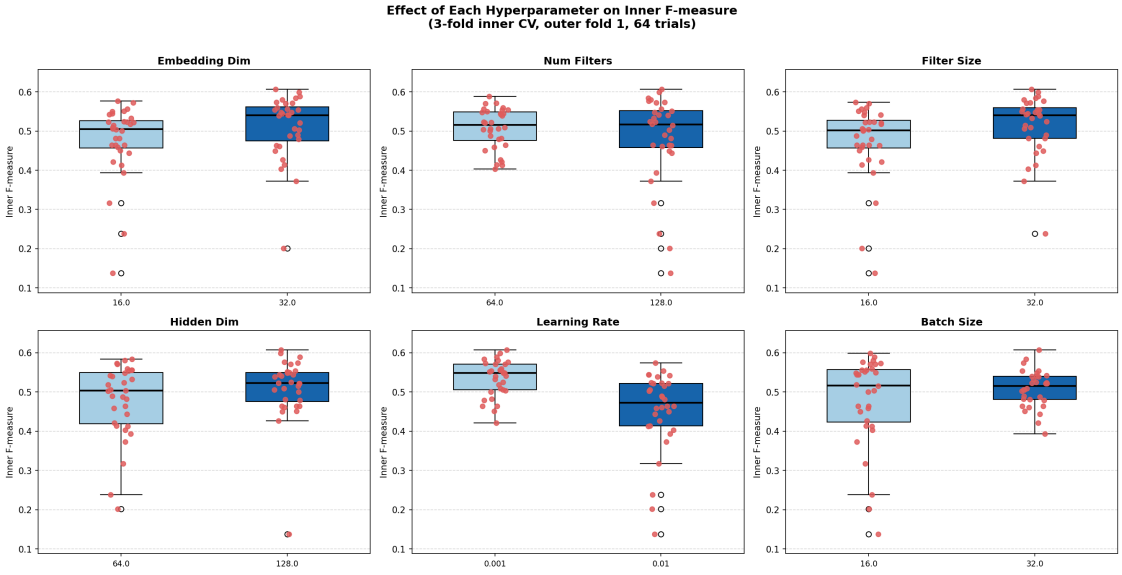

Figure 6.3: Effect of individual hyperparameters on Inner f-score

An analysis of pairwise interaction heatmaps reveal that the best inner F-measure of 0.607 is achieved under two configurations: learning rate 0.001 with 128 filters, and fil-

ter size 32 with hidden dimension 128. Learning rate remains the dominant factor, with the 0.001 setting consistently outperforming 0.01 by roughly 0.033–0.035 regardless of filter count. For the architectural parameters, filter size and hidden dimension behave additively and independently, with larger values in both jointly producing the highest scores without diminishing returns. Crucially, no antagonistic interactions are observed across either heatmap, and the performance surface is smooth and well-behaved, meaning that fixing the learning rate at 0.001 and selecting larger architectural values is a robust and sufficient strategy for optimising this CNN configuration.

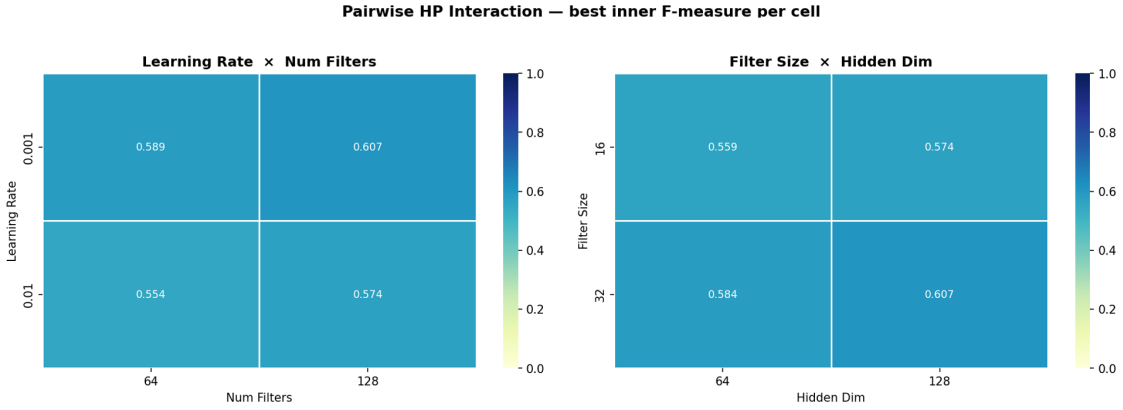

Figure 6.4: Pairwise hyperparameter interaction

Across all five outer folds, the hyperparameter tuning converged on a remarkably consistent configuration. Every fold selected a filter size of 32 with 128 filters and a learning rate of 0.001, suggesting these are important settings for this dataset. The embedding dimension varied mildly between 16 and 32, the hidden dimension alternated between 64 and 128, and the batch size was 16 in four of the five folds (32 only in fold 1). The inner F1 scores ranged from 0.578 to 0.636, with fold 4 performing best and fold 2 weakest, giving a mean inner F1 of approximately 0.600 — indicating moderate but consistent classification performance during tuning.

Table 6.1: Best Hyperparameters Selected by Inner 3-Fold Cross-Validation

Outer Fold	Embedding Dimension	Number of Filters	Filter Size	Hidden layer size	Learning Rate	Batch Size	Inner F1 (3-fold CV)
1	32	128	32	128	0.001	32	0.6070
2	16	128	32	64	0.001	16	0.5776
3	16	128	32	64	0.001	16	0.5819
4	16	128	32	128	0.001	16	0.6355
5	32	128	32	64	0.001	16	0.6021

6.3 Performance Metrics

Ultimately, the configurations with the best inner weighted F1 score are retrained on the full outer training split for 50 epochs and evaluated on their respective held-out outer test fold. The resulting metrics are then aggregated across all five test folds to provide a statistically reliable estimate of generalization performance.

The aggregated performance metrics obtained from the nested 5-fold cross-validation evaluation of the MalwareCNN model show an overall accuracy of 0.6513, indicating that the model correctly classifies approximately 65% of the samples across the held-out test folds. The precision score of 0.7189 is the highest among the reported metrics, suggesting that when the model predicts the right class, it is relatively reliable and produces fewer false positives. In contrast, the recall score of 0.6513 indicates that the model identifies roughly two-thirds of the true positive instances, implying that some samples may still be misclassified. The resulting F1-measure of 0.6634 reflects the harmonic balance between precision and recall, demonstrating moderate but consistent classification performance.

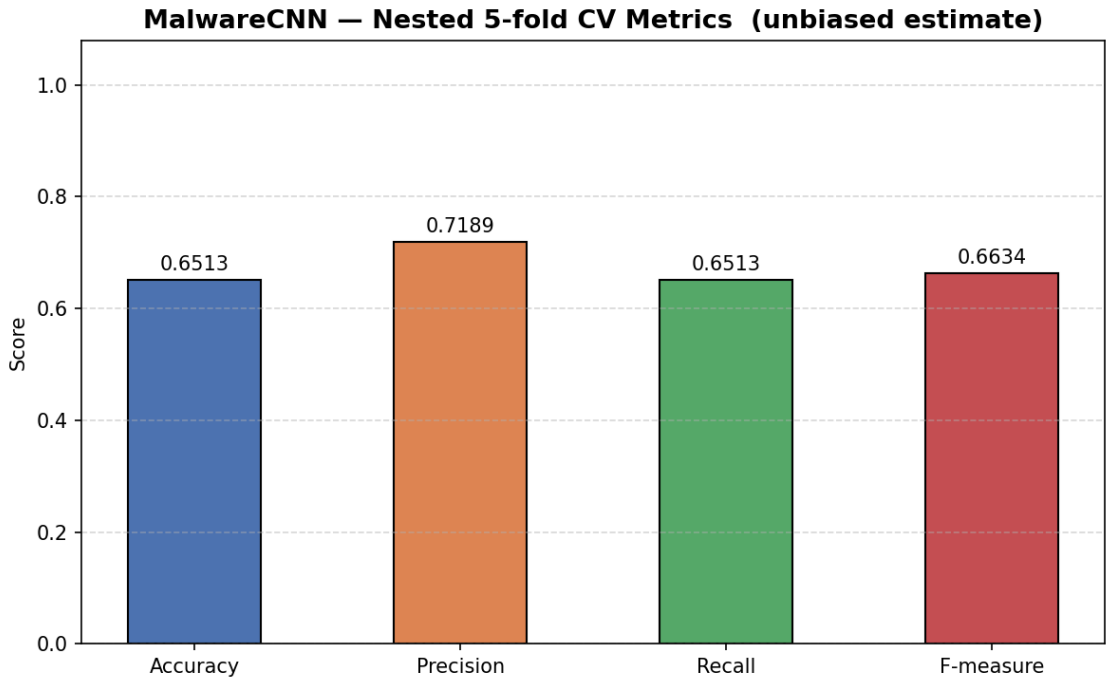

Figure 6.5: Final model's performance metrics

The training loss curves for the best hyperparameter configuration selected within each outer fold, trained for 50 epochs shows that, across all five folds, the cross-entropy loss decreases rapidly during the early epochs, dropping from values above 3.2 to below 1.0 within roughly the first 10–12 epochs, indicating that the model quickly learns useful fea-

ture representations from the input data. After approximately 30 epochs, the loss values begin to stabilize and gradually converge toward values close to 0, demonstrating consistent optimization behavior across folds. Minor fluctuations are observed in some folds during the mid-training phase, but these variations diminish as training progresses. The similar convergence patterns among all outer folds suggest that the training process is stable and that the selected hyperparameter configurations generalize consistently across different data partitions.

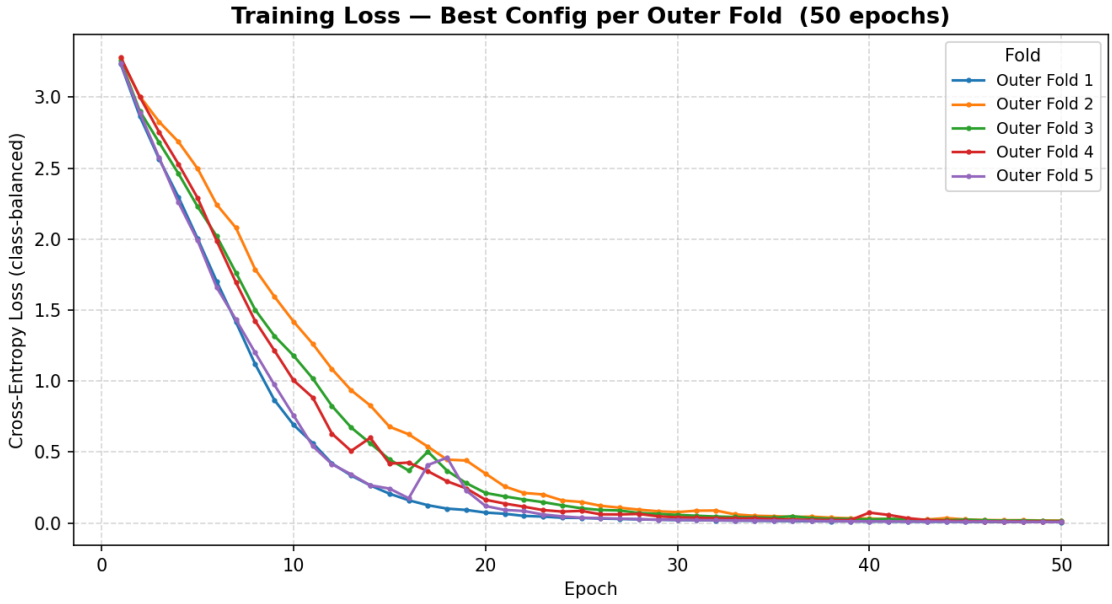

Figure 6.6: Final model's training loss curves

The per-class F1 scores indicate that the CNN achieves strong performance for several APT groups, with near-perfect results for Dust Storm and Poseidon Group (F1 = 1.00) and consistently high scores (0.85–0.94) for groups such as Lazarus Group, Strider, PROMETHIUM, and Machete, while a mid-tier of groups—including Stealth Falcon, Winnti Group, APT37, PLATINUM, Tropic Trooper, Whitefly, and ZIRCONIUM—show moderate performance (0.73–0.80). Lower scores (0.29–0.59) for groups like Carbanak, DarkHotel, Equation, APT18, APT38, Kimsuky, BlackTech, Threat Group-3390, and Lotus Blossom suggest higher confusion or weaker feature separability.

When compared with sample counts, there is a partial but not definitive correlation between dataset size and performance: larger classes such as Lazarus Group (23 samples), APT37 (18), PROMETHIUM (17), and Stealth Falcon (17) generally perform well, but exceptions exist, including Kimsuky (19) and Equation (12), which achieve only moderate F1 scores, and the heterogeneous "Other" class (34 samples), which remains around 0.50. Notably, some small classes like Dust Storm (8) and Poseidon Group (5) achieve perfect

scores, highlighting highly distinctive opcode patterns, while others with similar or larger sample sizes (e.g., Lotus Blossom and Threat Group-3390) perform poorly, indicating that class separability and feature distinctiveness play a more significant role than sample size alone in determining model performance.

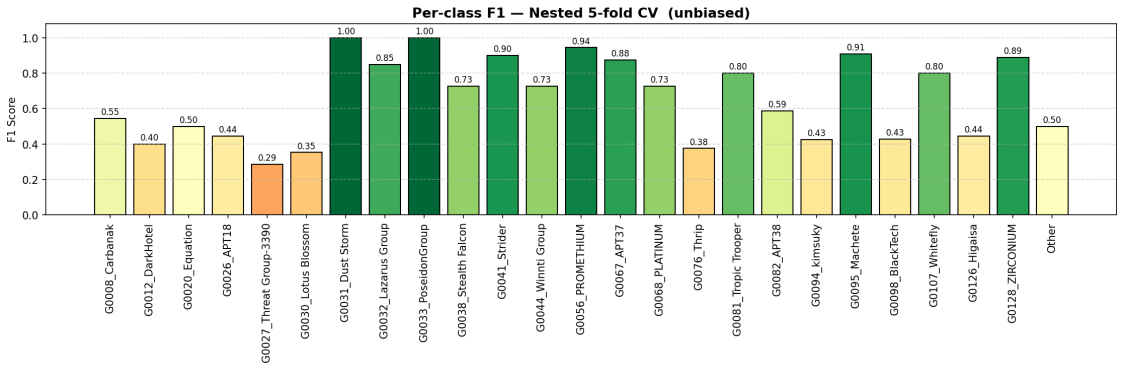

Figure 6.7: Final model's per class F1-score

6.4 Comparative Analysis

Compared with the previous models, the proposed MalwareCNN achieves the best performance in attributing malware opcodes to APT groups, with the highest accuracy (0.6513), precision (0.7189), recall (0.6513), and F1-score (0.6634). Among the classical methods, SVM with 2-gram features performs best but still trails the CNN, highlighting the advantage of deep feature learning over manual n-gram representations. Decision Trees and SVM with 1-grams show moderate performance, while KNN models consistently perform the worst.

Table 6.2: Model Performance Comparison

Model	**Accuracy**	**Precision**	**Recall**	**F1-score**
MalwareCNN	0.6513	0.7189	0.6513	0.6634
SVM (2-gram)	0.6322	0.6528	0.6322	0.6211
KNN (k=3) (2-gram)	0.4971	0.4965	0.4971	0.4765
KNN (k=4) (2-gram)	0.4932	0.4757	0.4932	0.4653
KNN (k=5) (2-gram)	0.4099	0.3683	0.4099	0.3746
Decision Tree (2-gram)	0.5632	0.5679	0.5632	0.5602
SVM (1-gram)	0.5747	0.6106	0.5747	0.5597
KNN (k=3) (1-gram)	0.4818	0.5223	0.4818	0.4696
KNN (k=4) (1-gram)	0.4712	0.4476	0.4712	0.4411
KNN (k=5) (1-gram)	0.4079	0.4384	0.4079	0.3948
Decision Tree (1-gram)	0.5632	0.5702	0.5632	0.5570

This comparison can also be visualized using the grouped bar chart shown below, which

illustrates the performance of each model across the evaluated metrics.

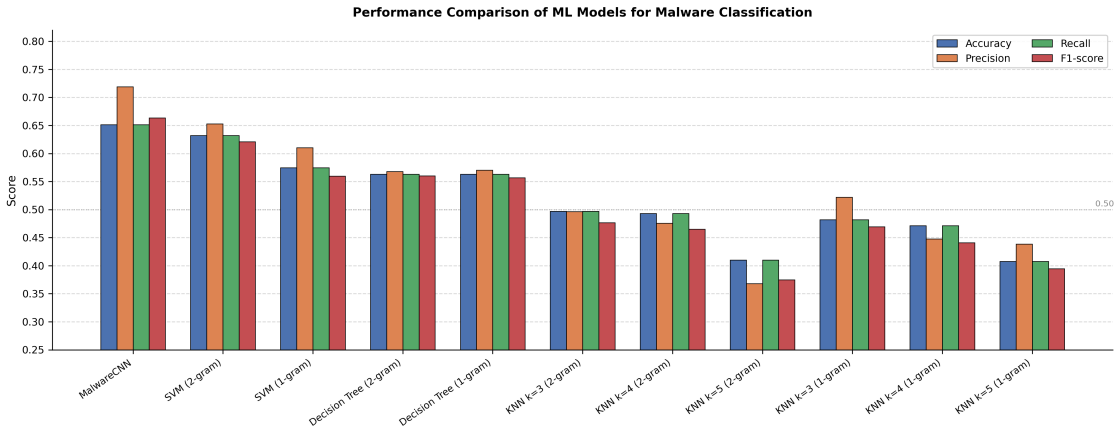

Figure 6.8: Final model's per class F1-score

Overall, these results highlight the effectiveness of the CNN architecture in capturing complex opcode patterns and demonstrate that 2-gram representations generally yield better results than 1-gram features for traditional models, though neither matches the CNN's performance.

7. Conclusion

This work evaluates a one-dimensional Convolutional Neural Network (CNN) for opcode-based malware classification under a multi-class APT attribution setting, extending prior binary-focused approaches to a more challenging and realistic formulation. Through controlled experimentation using nested cross-validation, the CNN consistently outperforms traditional machine learning baselines, including SVM, KNN, and Decision Trees, demonstrating the effectiveness of hierarchical feature extraction directly from raw opcode sequences over handcrafted representations.

Empirical analysis further indicates that model performance is highly sensitive to hyperparameter configuration, particularly learning rate and filter size, with improper settings leading to convergence instability. While the CNN successfully captures discriminative structural patterns, its performance remains constrained by dataset limitations such as class imbalance, vocabulary heterogeneity, and inter-class similarity.

Overall, these findings reinforce the suitability of deep learning architectures for opcode-driven malware classification, while also emphasizing the inherent complexity of scalable and robust multi-class APT attribution.

8. Future Work

Future work should prioritize overcoming representational and scalability constraints inherent in the current design. The imposed upper bound on sequence length (MAX_SEQ_LEN = 50,000) leads to systematic truncation of long opcode sequences, potentially eliminating temporally distant yet semantically critical instruction patterns; this motivates the exploration of architectures that inherently support long-range dependencies, such as transformer-based encoders with sparse attention or hierarchical sequence modeling frameworks. Furthermore, the limited dataset size and class imbalance introduce high variance in parameter estimation and restrict generalization, particularly for deep CNN-based models, necessitating the use of larger and more diverse corpora, along with self-supervised pretraining and data augmentation strategies. The heterogeneity of opcode vocabularies across multiple instruction set architectures further increases sparsity, motivating the investigation of opcode abstraction and embedding compression techniques. In addition, robustness to code obfuscation and adversarial perturbations remains unaddressed, suggesting the need for adversarially resilient training paradigms. Finally, incorporating explainability mechanisms (e.g., attribution over opcode sequences) and multimodal feature fusion with complementary static or dynamic analysis signals could significantly enhance both the interpretability and discriminative capability of the proposed framework in complex multi-class APT attribution settings.

References

[1] N. McLaughlin, J. Martinez del Rincon, B. Kang, S. Yerima, P. Miller, S. Sezer, Y. Safaei, E. Trickel, Z. Zhao, A. Doupé, and G.-J. Ahn, "Deep Android malware detection," in *Proc. 7th ACM Conf. on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 301–308.

[2] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, "A survey on automated dynamic malware-analysis techniques and tools," *ACM Computing Surveys*, vol. 44, no. 2, pp. 1–42, 2012.

[3] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *Computers & Security*, vol. 81, pp. 123–147, 2019.

[4] I. Santos, F. Brezo, J. Nieves, Y. K. Penya, B. Sanz, C. Laorden, and P. G. Bringas, "Idea: Opcode-sequence-based malware detection," in *Proc. Int. Symp. Engineering Secure Software and Systems (ESSoS)*, 2010, pp. 35–43.

[5] Z. Salehi, A. Sami, and M. Ghiasi, "MAAR: Robust features to detect malicious activity based on API calls, their arguments and return values," *Engineering Applications of Artificial Intelligence*, vol. 59, pp. 93–102, 2017.

[6] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2013, pp. 3111–3119.

[7] Y. Bengio, R. Ducharme, P. Vincent, and C. Janvin, "A neural probabilistic language model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003.

[8] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[9] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751.

[10] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. Int. Conf. Machine Learning (ICML)*, 2010, pp. 807–814.

[11] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, "Natural language processing (almost) from scratch," *Journal of Machine Learning Research*, vol. 12, pp. 2493–2537, 2011.

[12] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[13] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018.

[14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[15] S. Arlot and A. Celisse, "A survey of cross-validation procedures for model selection," *Statistics Surveys*, vol. 4, pp. 40–79, 2010.

[16] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.

[17] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[18] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

Appendices

Appendix 1: Confusion matrix for MalwareCNN